

Wolkite Unrnsity
College of Computing and Informatics
Department of Software Engineering

Introduction to Distributed System

Chapter 1: Introduction to Distributed Systems

February 22, 2025

Bekretsyon B(

Introduction

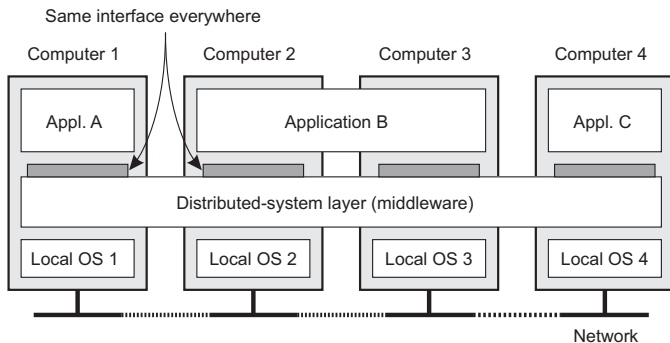
- Before the mid-80s, computers were
 - very expensive (hundred of thousands or even millions of dollars)
 - very slow (a few thousand instructions per second)
 - not connected among themselves
- After the mid-80s: two major developments
 - Cheap and powerful microprocessor-based computers appeared
 - Computer networks
 - LANs at speeds ranging from 10 to 1000 Mbps
 - WANs at speed ranging from 64 Kbps to gigabits/sec
- Consequence
 - feasibility of using a large network of computers to work for the same application; this is in contrast to the old centralized systems where there was a single computer with its peripherals

Distributed System: Definition

A distributed system is

*a collection of **autonomous computing elements** that appears to its users as a **single coherent system***

Two aspects: (1) independent computing elements and
(2) single system $\Rightarrow$ **middleware**.



Distributed System: Definition

Other Definitions

*A distributed system is a system designed to support the development of applications and services which can exploit a physical architecture consisting of multiple, autonomous processing elements that **do not share primary memory but cooperate by sending asynchronous messages over a communication network** (Blair & Stefani)*

Why Distributed?

- Resource and Data Sharing
 - Printers databases, multimedia servers, ...
- Availability, Reliability
 - The loss of some instances can be hidden
- Scalability, Extensibility
 - The system grows with demand (e.g., extra servers)
- Performance
 - Huge power (CPU, memory, ...) available
- Inherent distribution, communication
 - Organizational distribution, e-mail, video

Characteristics of Distributed Systems

- Differences between the computers and the ways they communicate are hidden from users
- Users and applications can interact with a distributed system in a consistent and uniform way regardless of location
- Distributed systems should be easy to expand and scale
- A distributed system is normally continuously available, even if there may be partial failures

Goals of Distributed Systems

To support **heterogeneous computers** and **networks** and to provide a **single-system view**,

A distributed system is often organized by means of a layer of software called **middleware** that extends over multiple machines.

A distributed system should

- easily connect users with resources (printers, computers, storage facilities, data, files, Web pages, ...)
 - **reasons**: economics, to collaborate and exchange information
- be transparent: hide the fact that the resources and processes are distributed across multiple computers
- be open
- be Scalable

Distribution transparency

A distributed system that is able to present itself to users and applications as if it were only a single computer system is said to be **transparent**

Transp.	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

Note

Distribution transparency is a nice a goal, but achieving it is a different story.

Distribution transparency

A distributed system that is able to present itself to users and applications as if it were only a single computer system is said to be **transparent**

Transp.	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

Note

Distribution transparency is a nice goal, but achieving it is a different story.

Degree of transparency

Observation

Aiming at full distribution transparency may be too much:

- Users may be located in **different continents**
- **Completely hiding failures** of networks and nodes is (theoretically and practically) **impossible**
 - You cannot distinguish a slow computer from a failing one
 - You can never be sure that a server actually performed an operation before a crash
- Full transparency will **cost performance**, exposing distribution of the system
 - Keeping Web caches **exactly** up-to-date with the master
 - Immediately flushing write operations to disk for fault tolerance

Degree of transparency

Observation

Aiming at full distribution transparency may be too much:

- Users may be located in **different continents**
- **Completely hiding failures** of networks and nodes is (theoretically and practically) **impossible**
 - You cannot distinguish a slow computer from a failing one
 - You can never be sure that a server actually performed an operation before a crash
- Full transparency will **cost performance**, exposing distribution of the system
 - Keeping Web caches **exactly** up-to-date with the master
 - Immediately flushing write operations to disk for fault tolerance

Degree of transparency

Observation

Aiming at full distribution transparency may be too much:

- Users may be located in **different continents**
- **Completely hiding failures** of networks and nodes is (theoretically and practically) **impossible**
 - You cannot distinguish a slow computer from a failing one
 - You can never be sure that a server actually performed an operation before a crash
- Full transparency will **cost performance**, exposing distribution of the system
 - Keeping Web caches **exactly** up-to-date with the master
 - Immediately flushing write operations to disk for fault tolerance

Degree of transparency

Observation

Aiming at full distribution transparency may be too much:

- Users may be located in **different continents**
- **Completely hiding failures** of networks and nodes is (theoretically and practically) **impossible**
 - You cannot distinguish a slow computer from a failing one
 - You can never be sure that a server actually performed an operation before a crash
- Full transparency will **cost performance**, exposing distribution of the system
 - Keeping Web caches **exactly** up-to-date with the master
 - Immediately flushing write operations to disk for fault tolerance

Openness of distributed systems

Open distributed system

Be able to interact with services from other open systems, irrespective of the underlying environment:

- Systems should conform to well-defined **interfaces**
- Systems should support **portability** of applications
 - components work on different platforms
- Systems should easily **interoperate**
 - components of different origin can communicate
- another goal of an open distributed system is that it should be **flexible** and **extensible**; easy to configure the system out of different components; easy to add new components, replace existing ones

Openness of distributed systems — Continued

- An **Open Distributed System** is a system that offers services according to **standard rules** that describe the syntax and semantics of those services; e.g., protocols in networks
- Standards - a necessity
 - should allow competition in non-normative areas
- In distributed systems, such services are often specified through interfaces often described using an **Interface Definition Language (IDL)**
 - specify only syntax: the names of the functions, types of parameters, return values, possible exceptions, ...

Scale in distributed systems

Observation

Many developers of modern distributed system easily use the adjective “scalable” without making clear **why** their system actually scales.

Scalability

a distributed system should be scalable:

- **Size:** adding more users and resources to the system
- **Geographically:** users and resources may be far apart
- **Administratively:** should be easy to manage even if it spans many administrative organizations

Observation

Most systems account only, to a certain extent, for size scalability. The (non)solution: powerful servers. Today, the challenge lies in geographical and administrative scalability.

Scale in distributed systems

Observation

Many developers of modern distributed system easily use the adjective “scalable” without making clear **why** their system actually scales.

Scalability

a distributed system should be scalable:

- **Size:** adding more users and resources to the system
- **Geographically:** users and resources may be far apart
- **Administratively:** should be easy to manage even if it spans many administrative organizations

Observation

Most systems account only, to a certain extent, for size scalability. The (non)solution: powerful servers. Today, the challenge lies in geographical and administrative scalability.

Scale in distributed systems

Observation

Many developers of modern distributed system easily use the adjective “scalable” without making clear **why** their system actually scales.

Scalability

a distributed system should be scalable:

- **Size:** adding more users and resources to the system
- **Geographically:** users and resources may be far apart
- **Administratively:** should be easy to manage even if it spans many administrative organizations

Observation

Most systems account only, to a certain extent, for size scalability. The (non)solution: powerful servers. Today, the challenge lies in geographical and administrative scalability.

Scale in distributed systems

Scalability problems: performance problems caused by limited capacity of servers and networks

Concept	Example
Centralized services	Single server for all users-mostly for security reasons
Centralized data	A single on-line telephone book
Centralized algorithms	Doing routing based on complete information

● Scaling Techniques

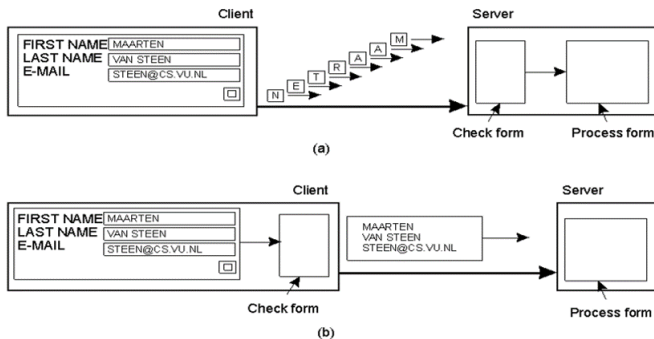
- How to solve scaling problems
- The problem is mainly performance, and arises as a result of limitations in the capacity of servers and networks (for geographical scalability)
- Three possible solutions: **hiding communication latencies**, **distribution**, and **replication**

Techniques for scaling

Hide communication latencies

- Try to avoid waiting for responses to remote service requests
- Let the requester do other useful job
- i.e., construct requesting applications that use only **asynchronous** communication instead of **synchronous communication**; when a reply arrives the application is interrupted
- Good for batch processing and parallel applications but not for interactive applications
- For interactive applications, move part of the job to the client to reduce communication; e.g. filling a form and checking the entries

Techniques for scaling



- a server checking the correctness of field entries
- a client doing the job

e.g., shipping code is now supported in Web applications using Java Applets

Techniques for scaling

Distribution

Partition data and computations across multiple machines:

- Move computations to clients (Java applets)
- Decentralized naming services (DNS)
- Decentralized information systems (WWW)

Techniques for scaling

Replication/caching

- Replicate components across a distributed system to increase **availability** and for **load balancing**, leading to better performance
 - Replicated file servers and databases
 - Mirrored Web sites
- decided by the owner of a resource
- **caching (a special form of replication)** also reduces communication latency; decided by the user
- but, caching and replication may lead to consistency problems (see Chapter 6 - Consistency and Replication)
 - Web caches (in browsers and proxies)
 - File caching (at server and client)

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Types of distributed systems

- Distributed computing systems
 - Focus on computation
 - Goal: High performance computing tasks
- Distributed information systems
 - Focus on interoperability (the ability to exchange and use information)
 - Goal: Distribute information across several servers
- Distributed pervasive systems
 - Focus on mobile, embedded, communicating systems
 - Goal: Spread a real-life environment with a large variety of smart devices

Distributed computing systems

Distributed Computing systems

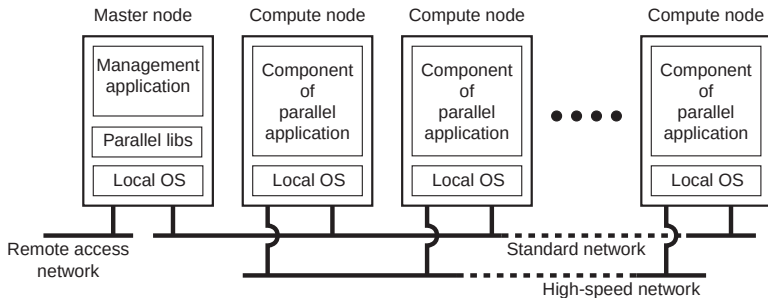
- Used for high-performance computing tasks two types: **cluster computing** and **grid computing**
- Many distributed systems are configured for **High-Performance Computing**

Cluster Computing

Essentially a group of high-end systems connected through a LAN:

- A collection of similar workstations or PCs (homogeneous), closely connected by means of a high-speed LAN
- Each node runs the same operating system
- Used for parallel programming in which a single compute intensive program is run in parallel on multiple machines

Distributed computing systems



An example of a cluster computing system

Distributed computing systems

Grid Computing

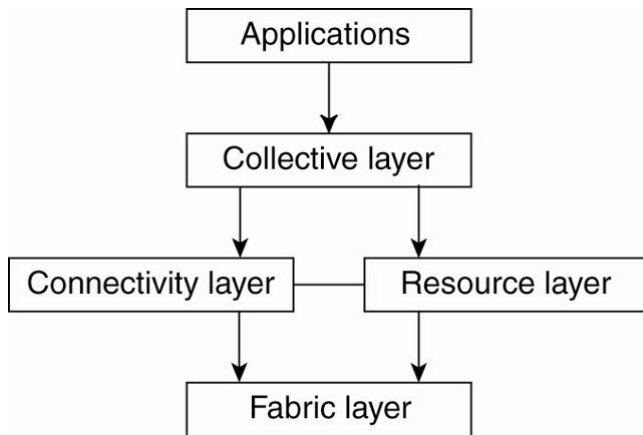
Lots of nodes (including clusters across multiple subnets) from everywhere:

- Resource sharing and coordinated problem solving in dynamic, multi-institutional virtual organizations (I. Foster)
- High degree of heterogeneity: no assumptions are made concerning hardware, operating systems, networks, administrative domains, security policies, etc.
- **Key issue in a grid computing system** - resources from different organizations are brought together to allow the collaboration of a group of people or institutions - forms a virtual organization.

Note

To allow for collaborations, grids generally use **virtual organizations**. In essence, this is a grouping of users (or better: their IDs) that will allow for authorization on resource allocation.

Distributed computing systems



A layered architecture for grid computing systems.

Distributed computing systems

- **Fabric layer** - provides interfaces to local resources at a specific site.
 - interfaces are designed to allow sharing of resources within a virtual organization.
 - provide functions for querying the state and capabilities of a resource, along with functions for actual resource management (e.g., locking resources).
- **Connectivity layer** - consists of communication protocols for supporting grid transactions that span the usage of multiple resources.
 - contains security protocols to authenticate users and resources.
 - in many cases human users are not authenticated - programs acting on behalf of the users are authenticated.
- **Resource layer** - responsible for managing a single resource.
 - uses functions provided by the connectivity layer and calls directly the interfaces made available by the fabric layer.
 - responsible for access control, and hence will rely on the authentication performed as part of the connectivity layer.
- **Collective layer** - handles access to multiple resources
 - consists of services for resource discovery, allocation and scheduling of tasks onto multiple resources, data replication, etc.

Distributed Information Systems

- Evolved in organizations that were confronted with a wealth of networked applications, but for which **interoperability** turned out to be problematic.
- Many of the existing middleware solutions are the result of working with an infrastructure in which it was easier to integrate applications into an enterprise-wide information system

Distributed Information Systems

Several levels at which integration took place:

- ❶ A networked application simply consisted of a server running that application (often including a database) and making it available to remote programs, called clients.
 - Clients could send a request to the server for executing a specific operation, after which a response would be sent back. Integration at the lowest level would allow clients to wrap a number of requests, possibly for different servers, into a single larger request and have it executed as a distributed transaction.
 - The key idea was that all, or none of the requests would be executed.
- ❷ As applications became more sophisticated and were gradually separated into independent components (notably distinguishing database components from processing components), it became clear that integration should also take place by letting applications communicate directly with each other.

Distributed Information Systems

Results: two forms of distributed systems

- Transaction processing systems
- Enterprise application integration (EAI)

Transaction processing system

- Focus on database applications - operations on a database are usually carried out in the form of transactions.
- Programming using transactions requires special primitives that must either be supplied by the underlying distributed system or by the language runtime system.
- Example primitives for transactions

Distributed Information Systems : Transactions

Primitive	Description
BEGIN_TRANSACTION	Mark the start of a transaction
END_TRANSACTION	Terminate the transaction and try to commit
ABORT_TRANSACTION	Kill the transaction and restore the old values
READ	Read data from a file, a table, or otherwise
WRITE	Write data to a file, a table, or otherwise

- Exact list of primitives depends on what kinds of objects are being used in the transaction.
 - e.g. mail system - may be primitives to send, receive, and forward mail.
 - e.g. accounting system - may be different.
- Ordinary statements, procedure calls, etc, are allowed inside a transaction.
- Remote procedure calls (RPCs) procedure calls to remote servers, are often also encapsulated in a transaction, leading to what is known as a transactional RPC.

Distributed Information Systems : Transactions

- e.g., assume the following banking operation
 - withdraw an amount x from account 1
 - deposit the amount x to account 2
- What happens if there is a problem after the first activity is carried out?
- Group the two operations into one transaction; either both are carried out or neither
- We need a way to roll back when a transaction is not completed

Distributed Information Systems : Transactions

- e.g., assume the following banking operation
 - withdraw an amount x from account 1
 - deposit the amount x to account 2
- What happens if there is a problem after the first activity is carried out?
- Group the two operations into one transaction; either both are carried out or neither
- We need a way to roll back when a transaction is not completed

Distributed Information Systems : Transactions

E.g. reserving a seat from White Plains to Malindi through JFK and Nairobi airports

```
BEGIN_TRANSACTION  
reserve WP → JFK;  
reserve JFK → Nairobi;  
reserve Nairobi → Malindi;  
END_TRANSACTION
```

(a)

```
BEGIN_TRANSACTION  
reserve WP → JFK;  
reserve JFK → Nairobi;  
reserve Nairobi → Malindi full ⇒  
ABORT_TRANSACTION
```

(b)

(a) transaction to reserve three flights commits

(b) transaction aborts when third flight is unavailable

Distributed information systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (**ACID**)

Atomicity: All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.

Consistency: A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.

Isolation: Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.

Durability: After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Distributed information systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (**ACID**)

Atomicity: All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.

Consistency: A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.

Isolation: Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.

Durability: After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Distributed information systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (**ACID**)

Atomicity: All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.

Consistency: A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.

Isolation: Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.

Durability: After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Distributed information systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (**ACID**)

Atomicity: All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.

Consistency: A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.

Isolation: Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.

Durability: After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Distributed information systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (**ACID**)

- Atomicity:** All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.
- Consistency:** A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.
- Isolation:** Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.
- Durability:** After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Distributed information systems: Transactions

Classification of Transactions

- A transaction could be flat, nested or distributed

Flat Transaction

- Consists of a series of operations that satisfy the ACID properties
- Simple and widely used but with some limitations
 - Do not allow partial results to be committed or aborted
 - i.e., atomicity is also partly a weakness
 - In our airline reservation example, we may want to accept the first two reservations and find an alternative one for the last
- Some transactions may take too much time

Distributed information systems: Transactions

Nested Transaction

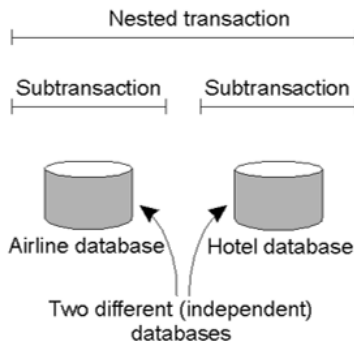
- Constructed from a number of sub transactions; it is logically decomposed into a hierarchy of sub transactions
- The top-level transaction forks off children that run in parallel, on different machines; to gain performance or for programming simplicity
- Each may also execute one or more sub transactions
- Permanence (durability) applies only to the top-level transaction; commits by children should be undone

Distributed information systems: Transactions

Distributed Transaction

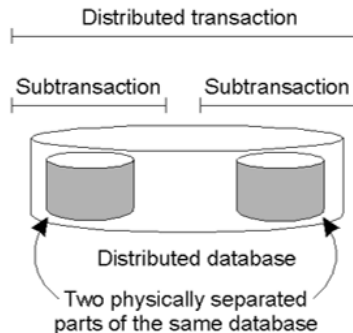
- A flat transaction that operates on data that are distributed across multiple machines
- **Problem:** separate algorithms are needed to handle the locking of data and committing the entire transaction; **see later in Chapter 8 for distributed commit**

Distributed information systems: Transactions



(a)

(a) a nested transaction



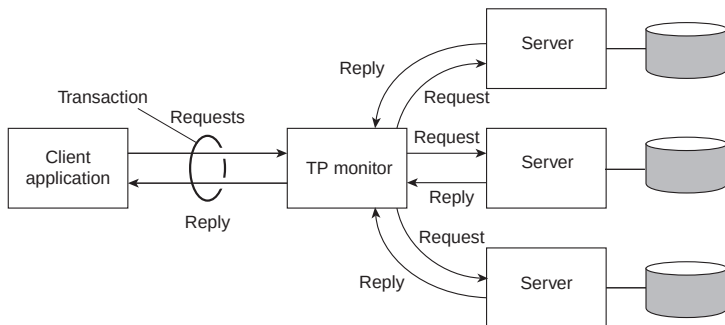
(b)

(b) distributed transaction

Transaction processing monitor

Observation

In many cases, the data involved in a transaction is distributed across several servers. A **TP Monitor** is responsible for coordinating the execution of a transaction



Distr. info. systems: Enterprise application integration

Problem

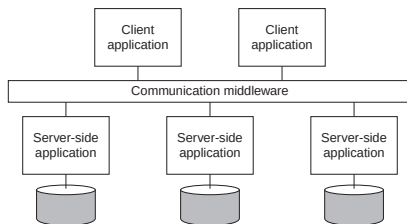
A TP monitor doesn't separate apps from their databases. Also needed are facilities for direct communication between apps.

- The more applications became decoupled from the databases they were built upon, the more evident it became that facilities were needed to integrate applications independent from their databases.
- Application components should be able to communicate directly with each other and not merely by means of the request/reply behavior that was supported by transaction processing systems.

Distr. info. systems: Enterprise application integration

Result:

Middleware as a communication facilitator in enterprise application integration.



There are different communication model:

- Remote Procedure Call (RPC)
- Remote Method Invocation (RMI)
- Message-Oriented Middleware (MOM)

Distributed pervasive systems

Observation

Emerging next-generation of distributed systems in which nodes are small, mobile, and often embedded in a larger system, characterized by the fact that the system **naturally blends into the user's environment**.

Three (overlapping) subtypes

- **Ubiquitous computing systems**: pervasive and **continuously present**, i.e., there is a continuous interaction between system and user.
- **Mobile computing systems**: pervasive, but emphasis is on the fact that devices are **inherently mobile**.
- **Sensor (and actuator) networks**: pervasive, with emphasis on the actual (collaborative) **sensing** and **actuation** of the environment.

Distributed pervasive systems

Observation

Emerging next-generation of distributed systems in which nodes are small, mobile, and often embedded in a larger system, characterized by the fact that the system **naturally blends into the user's environment**.

Three (overlapping) subtypes

- **Ubiquitous computing systems**: pervasive and **continuously present**, i.e., there is a continuous interaction between system and user.
- **Mobile computing systems**: pervasive, but emphasis is on the fact that devices are **inherently mobile**.
- **Sensor (and actuator) networks**: pervasive, with emphasis on the actual (collaborative) **sensing** and **actuation** of the environment.

Distributed pervasive systems

Observation

Emerging next-generation of distributed systems in which nodes are small, mobile, and often embedded in a larger system, characterized by the fact that the system **naturally blends into the user's environment**.

Three (overlapping) subtypes

- **Ubiquitous computing systems**: pervasive and **continuously present**, i.e., there is a continuous interaction between system and user.
- **Mobile computing systems**: pervasive, but emphasis is on the fact that devices are **inherently mobile**.
- **Sensor (and actuator) networks**: pervasive, with emphasis on the actual (collaborative) **sensing** and **actuation** of the environment.

Distributed pervasive systems

Observation

Emerging next-generation of distributed systems in which nodes are small, mobile, and often embedded in a larger system, characterized by the fact that the system **naturally blends into the user's environment**.

Three (overlapping) subtypes

- **Ubiquitous computing systems**: pervasive and **continuously present**, i.e., there is a continuous interaction between system and user.
- **Mobile computing systems**: pervasive, but emphasis is on the fact that devices are **inherently mobile**.
- **Sensor (and actuator) networks**: pervasive, with emphasis on the actual (collaborative) **sensing** and **actuation** of the environment.

Ubiquitous computing systems

Basic characteristics

- (**Distribution**) Devices are networked, distributed, and accessible in a transparent manner
- (**Interaction**) Interaction between users and devices is highly unobtrusive
- (**Context awareness**) The system is aware of a user's context in order to optimize interaction
- (**Autonomy**) Devices operate autonomously without human intervention, and are thus highly self-managed
- (**Intelligence**) The system as a whole can handle a wide range of dynamic actions and interactions

Ubiquitous computing systems

Basic characteristics

- (**Distribution**) Devices are networked, distributed, and accessible in a transparent manner
- (**Interaction**) Interaction between users and devices is **highly unobtrusive**
- (**Context awareness**) The system is **aware** of a **user's context** in order to optimize interaction
- (**Autonomy**) Devices **operate autonomously without human intervention**, and are thus highly self-managed
- (**Intelligence**) The system as a whole can handle a wide range of dynamic actions and interactions

Ubiquitous computing systems

Basic characteristics

- (**Distribution**) Devices are networked, distributed, and accessible in a transparent manner
- (**Interaction**) Interaction between users and devices is highly unobtrusive
- (**Context awareness**) The system is aware of a user's context in order to optimize interaction
- (**Autonomy**) Devices operate autonomously without human intervention, and are thus highly self-managed
- (**Intelligence**) The system as a whole can handle a wide range of dynamic actions and interactions

Ubiquitous computing systems

Basic characteristics

- (**Distribution**) Devices are networked, distributed, and accessible in a transparent manner
- (**Interaction**) Interaction between users and devices is highly unobtrusive
- (**Context awareness**) The system is aware of a user's context in order to optimize interaction
- (**Autonomy**) Devices operate autonomously without human intervention, and are thus highly self-managed
- (**Intelligence**) The system as a whole can handle a wide range of dynamic actions and interactions

Ubiquitous computing systems

Basic characteristics

- (**Distribution**) Devices are networked, distributed, and accessible in a transparent manner
- (**Interaction**) Interaction between users and devices is highly unobtrusive
- (**Context awareness**) The system is aware of a user's context in order to optimize interaction
- (**Autonomy**) Devices operate autonomously without human intervention, and are thus highly self-managed
- (**Intelligence**) The system as a whole can handle a wide range of dynamic actions and interactions

Mobile computing systems

Observation

Mobile computing systems are generally a subclass of ubiquitous computing systems and meet all of the five requirements.

Typical characteristics

- Many different types of mobile devices: smart phones, remote controls, car equipment, and so on
- Wireless communication
- Devices may continuously change their location $\Rightarrow$
 - setting up a route may be problematic, as routes can change frequently
 - devices may easily be temporarily disconnected $\Rightarrow$ **disruption-tolerant networks**

Sensor networks

Characteristics

The **nodes** to which sensors are attached are:

- Many (10s-1000s)
- Simple (small memory/compute/communication capacity)
- Often battery-powered (or even battery-less)

Sensor networks as distributed systems

